PROJECT EXPLANATION & USER GUIDE

SachetGuard PM

A simple guide to what the app does, how to use each screen, and how the code is put together.

Project: Development of a Predictive/Preventive Maintenance Model for Selected Components in Production Equipment – A Case Study of a Sachet Water Production Company, Makurdi, Benue State, Nigeria.
Student: Elaigwu Wisdom Daniel · 21/57894/UE
Department: Mechanical Engineering, JOSTUM · 2026

CONTENTS

What is inside

01 The project in one page
The problem, the idea and what the app gives you

02 Getting started
How to open the app and the buttons at the top

03 A tour of every screen
What each page shows and what you can do on it

04 How the app thinks
The six steps from failure records to a maintenance plan

05 Results with the sample data
The numbers the app gives for the three parts

06 How the code is put together
Every file, what it does, and how they work together

07 Entering real factory data
Step-by-step, to replace the sample data

08 Tips for the defence
Likely questions and simple answers

09 Limits of the work and glossary
What to say honestly, and the meaning of each term

01 The project in one page

SachetGuard PM is a computer program that tells a sachet water factory **when to replace key machine parts before they break**, and how much money this saves.

The problem

A sachet water filling and sealing machine has parts that wear out. This project looks at three of them:

Electric motor
Drives the machine. Too much heat damages its winding insulation.

Bearing system
Lets shafts turn smoothly. Wear shows up as heat, noise and vibration.

Heat sealing element
Seals each sachet. It burns out after many hours of heating.

When one of these parts breaks without warning, production stops. The factory pays for the repair **and** loses the water it could have sold during the stop. Today, most small factories wait for parts to break and then fix them. This is called **reactive** or **run-to-failure** maintenance.

The idea

Every part has a pattern of how long it usually lasts. If we record the hours at which parts failed in the past, we can use mathematics to learn that pattern. Then we can replace a part **just before** it is likely to fail. Not too early, because that wastes a good part. Not too late, because that causes a breakdown.

> **In one sentence:** the app learns from past failures, checks each part's current condition (hours, temperature and vibration), and tells you the cheapest time to replace it.

What the app gives you

- A **health status** for each part: Healthy Monitor Critical.
- The **remaining life** of each part, in running hours and in days.
- The **chance the part will fail** in the next 100, 200 and 500 hours.
- The **best replacement age** (called **T***), the age that gives the lowest cost.
- A **12-month maintenance plan** that can be printed.
- A **cost comparison** in Naira: waiting for breakdowns against planned replacement.
- A **validation check**: would the plan have prevented real past failures?

Why it is easy to use

- It needs **no installation and no internet**. You open one file in a web browser.
- Your data **stays on your own computer**.
- Every result is written in **plain English**, with a short recommendation.
- Every formula can be shown with the **real numbers filled in**, so the work can be checked by hand.

02 Getting started

You only need a web browser such as Chrome, Edge, Firefox or Safari. It works on a laptop and on a phone.

Opening the app

1. Open the project folder.
2. **Double-click `index.html`.** The app opens in your browser.
3. The app starts with **sample data**, so every screen works straight away. A yellow stripe at the top reminds you of this.

> **One-file version.** The file `dist/sachetguard-standalone.html` holds the whole app in a single file. It is easy to email, copy to a USB stick or open on another computer.

The buttons at the top

BUTTON	WHAT IT DOES
Guided demo	Walks through the main screens with a short caption on each. Use Next and Back, or press Esc to leave. Good for presentations.
Presentation mode (screen icon)	Makes text and charts larger so they are easy to read on a projector.
Theme (sun / moon icon)	Switches between light and dark colours.

Moving around

On a computer, the menu is on the left. It has three groups:

- **Components**: one link for each part. The coloured dot shows its status. The most urgent part is always at the top. The **+** adds a new part.
- **Tools**: Model Calibration, Maintenance Schedule, Cost Comparison and Validation.
- **App**: Settings and How It Works.

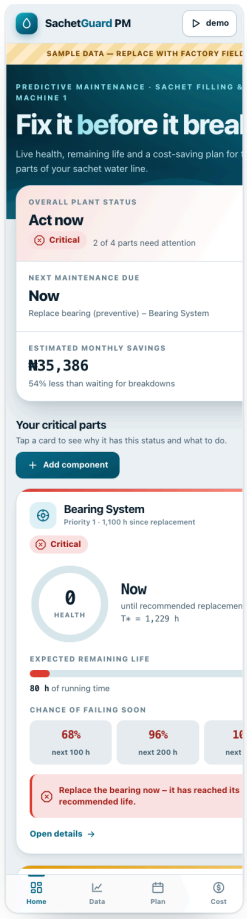
On a phone, the menu moves to a **bar at the bottom of the screen** with Home, Data, Plan, Cost and More.

Where your data is kept

Everything you type is saved automatically inside the browser on that device. Nothing is sent to the internet.

Because of this, data typed on one computer will not appear on another. To move your data, or to keep a safe copy, use **Settings → Back up data**. This downloads one file. Use **Restore backup** to load it again.

> **Tip:** make a backup after entering real data, and before clearing the browser's history.

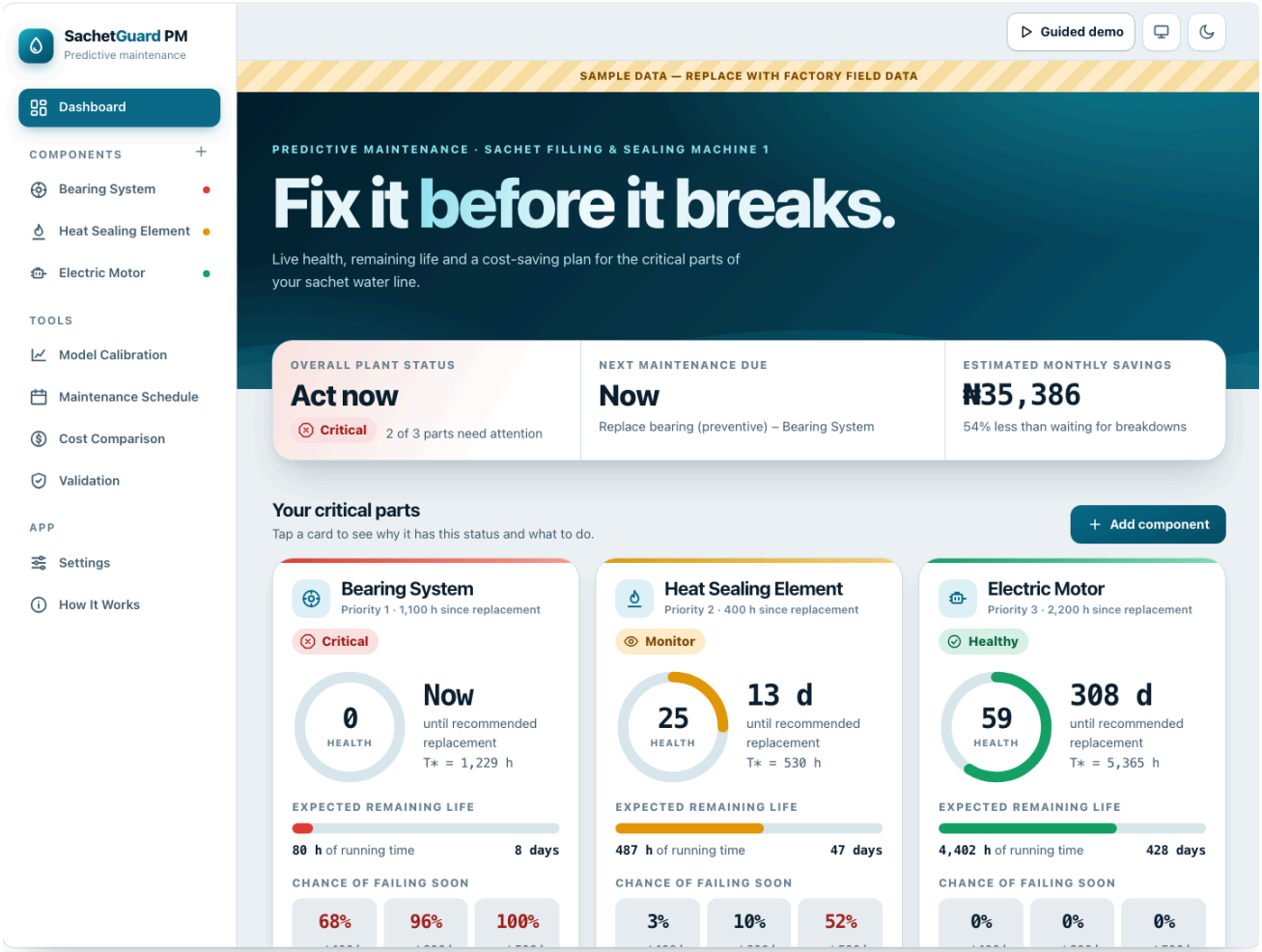


Phone view. The same app on a small screen.

03 A tour of every screen

Each page below has a picture, what the page shows, and what you can do on it. The pictures use the sample data.

3.1 Dashboard



Dashboard. The first page you see.

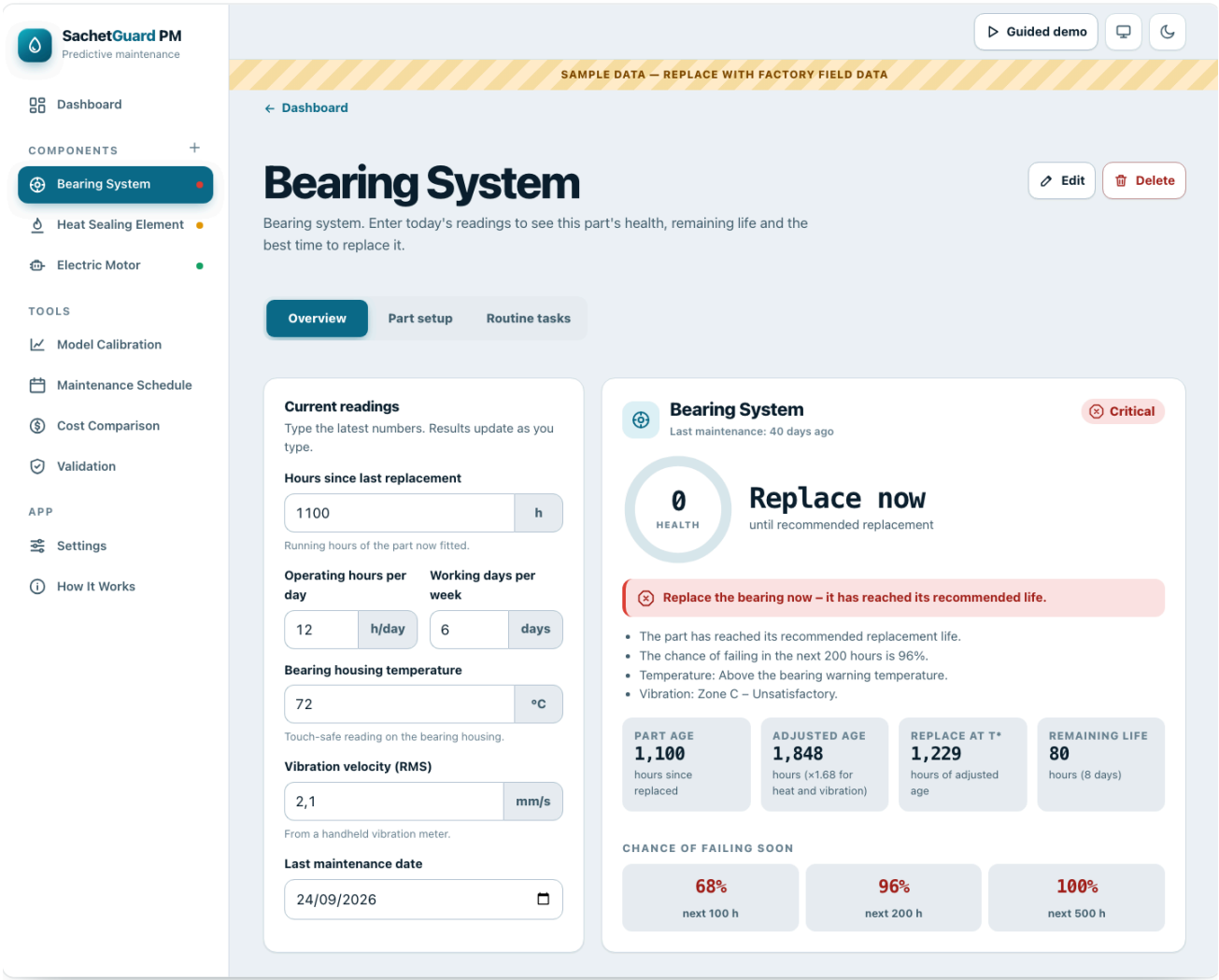
WHAT IT SHOWS

- **Overall plant status**: the worst status of all parts.
- **Next maintenance due**: the next job in the plan.
- **Estimated monthly savings** from planned replacement.
- One card per part, with its health score (0–100), days until replacement and remaining life. The most urgent part comes first.

WHAT YOU CAN DO

- Click a card to open that part's page.
- Press **Add component** to add a new machine part.

3.2 Component page – Overview



Bearing System, Overview tab. Readings on the left, results on the right.

WHAT IT SHOWS

- The **status**, a **recommendation** in one sentence, and the reasons for the status.
- **Part age** and **adjusted age** (age after heat and vibration are taken into account).
- The **replacement age T*** and the **remaining life**.
- The **chance of failing** in the next 100, 200 and 500 hours.

WHAT YOU CAN DO

- Type today's readings: hours since the part was last replaced, temperature, vibration and the last maintenance date.
- Every result **updates as you type**.
- Use **Edit** or **Delete** to change or remove the part.

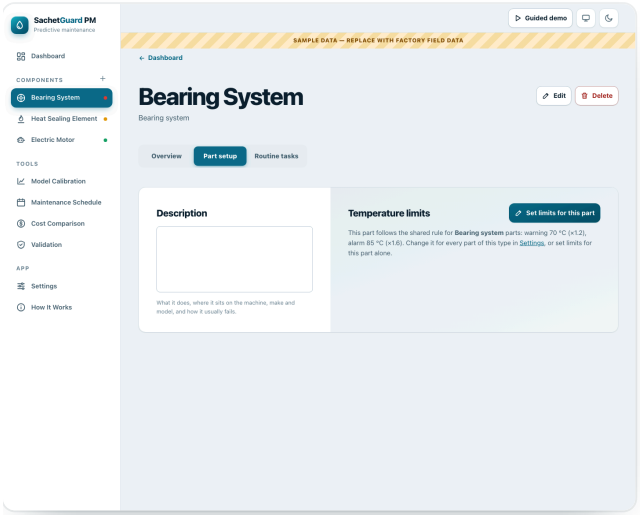
3.2 (continued) – The three charts



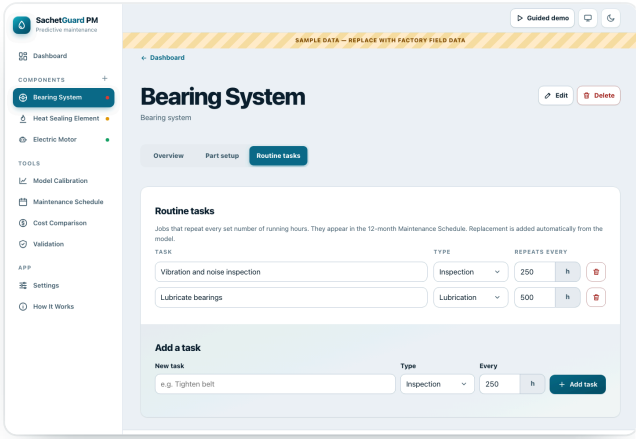
Charts on the component page. The purple "You are here" line marks the part's adjusted age today.

- **How likely is the part to still be working?** The teal line is the wear-out model learned from the data. The dashed amber line assumes failures happen at random. The red dashed line is the best replacement age.
- **How risky is the next hour?** A line that rises means the part is wearing out. The older it is, the more likely it fails soon.
- **What does each running hour cost?** Replacing too early wastes parts. Replacing too late causes breakdowns. The red dot at the bottom of the curve is the cheapest replacement age, T*.
- Below the charts, **"How was this calculated?"** opens every formula with the real numbers filled in. This is very useful when an examiner asks how a number was found.

3.3 Component page – Part setup and Routine tasks



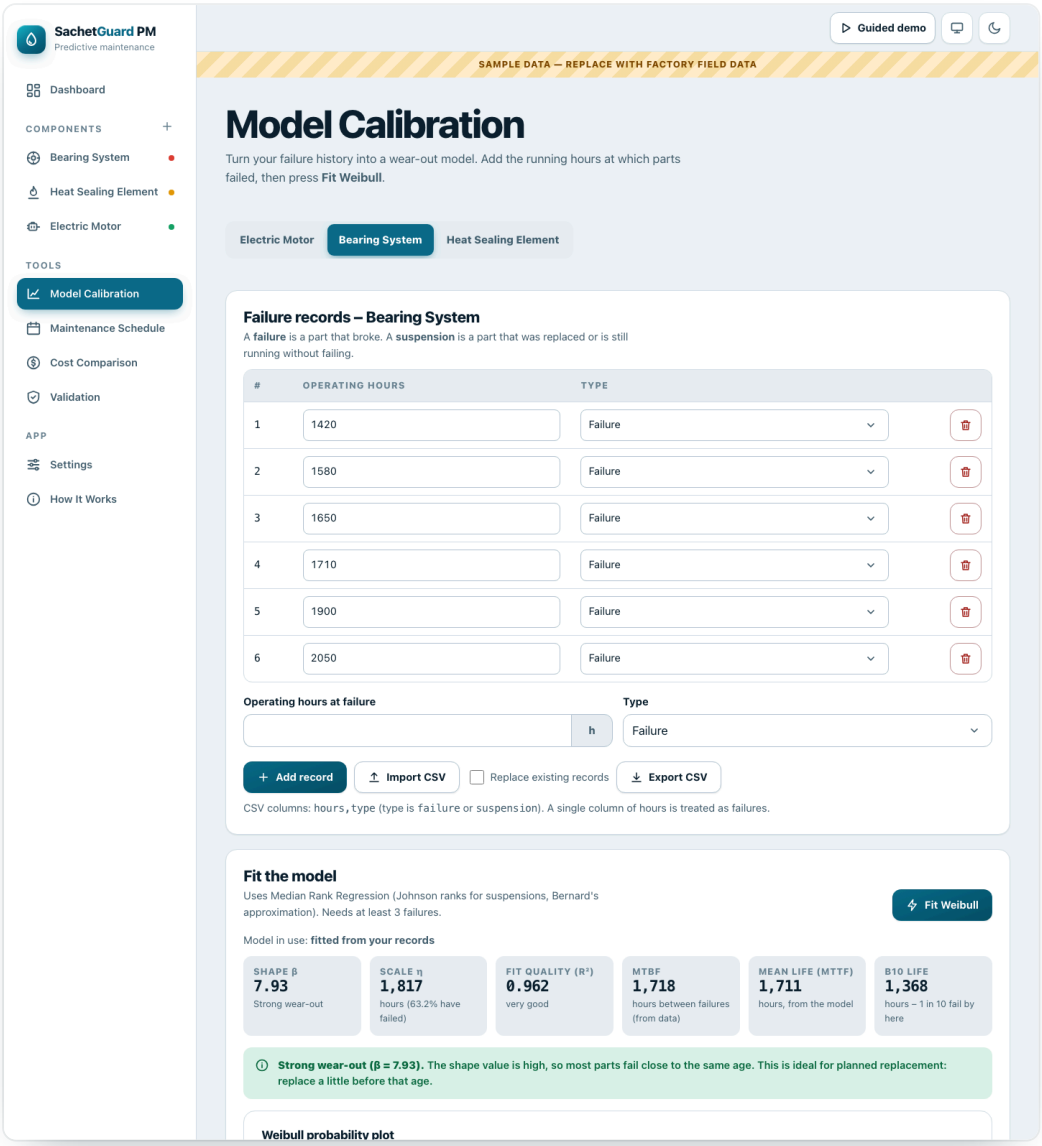
Part setup. Description and temperature limits.



Routine tasks. Jobs that repeat.

- **Part setup** lets you write a short description of the part. You can also give this part its **own warning and alarm temperatures**, with a note saying where the limits came from (for example, the maker's manual). If you do not, the part uses the shared limits in Settings.
- **Routine tasks** lists jobs such as inspection, lubrication and cleaning. Each job repeats after a set number of running hours. You can add, rename or remove jobs. They appear in the Maintenance Schedule automatically.

3.4 Model Calibration



Model Calibration. Failure records at the top, the fitted model below.

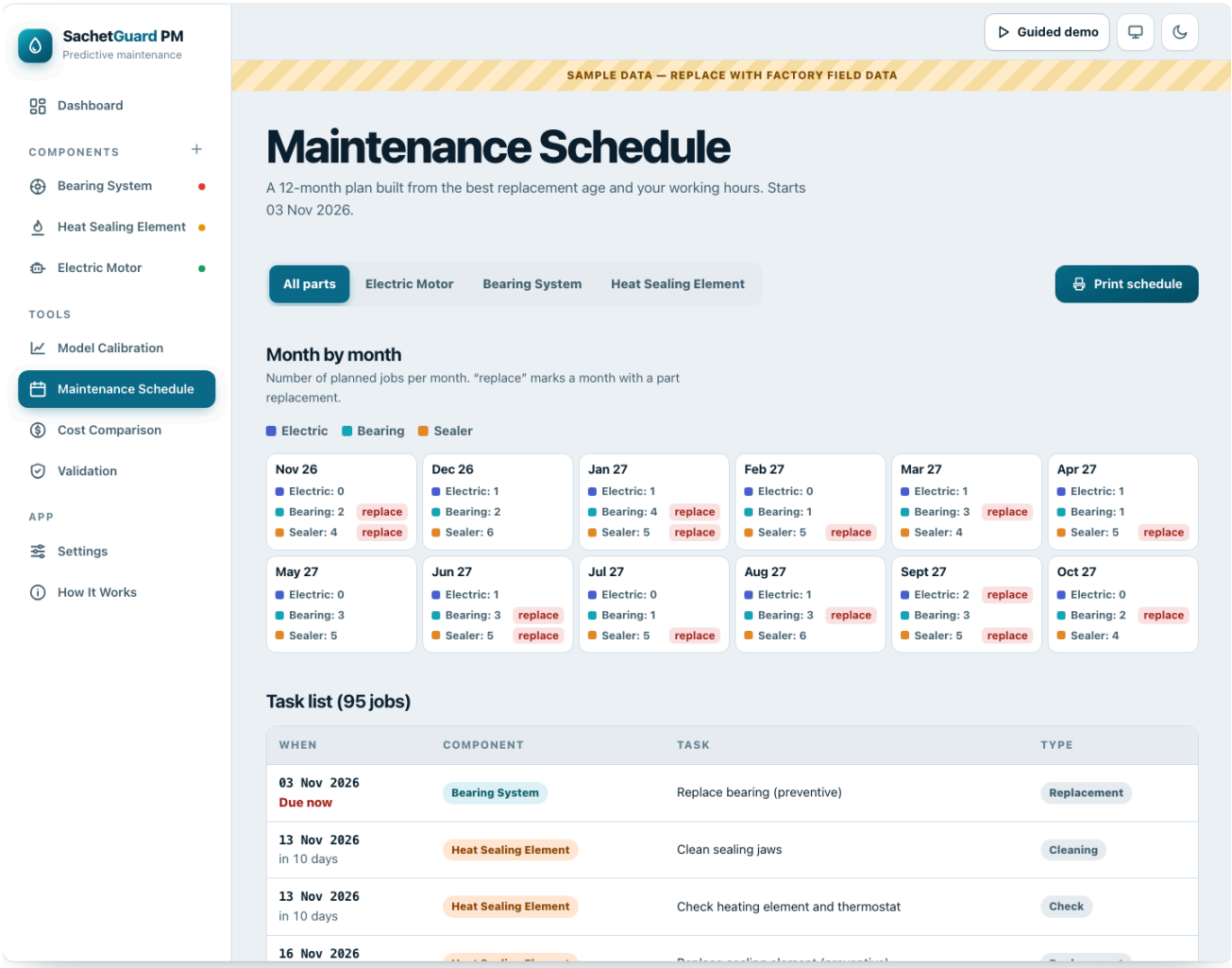
WHAT IT SHOWS

- The **failure records** for the chosen part.
- The fitted **shape (β)** and **scale (η)**, the fit quality (R^2), MTBF, mean life and B10 life.
- A plain sentence explaining what β means for this part.
- A **Weibull plot**. If the dots follow the straight line, the model fits the data well.

WHAT YOU CAN DO

- Add a record: the running hours when a part failed.
- Mark a record as a **suspension** if the part was removed or is still running without failing.
- **Import CSV** or **Export CSV** to move many records at once.
- Press **Fit Weibull** to build the model. At least 3 failures are needed.

3.5 Maintenance Schedule



Maintenance Schedule. A 12-month plan, built automatically.

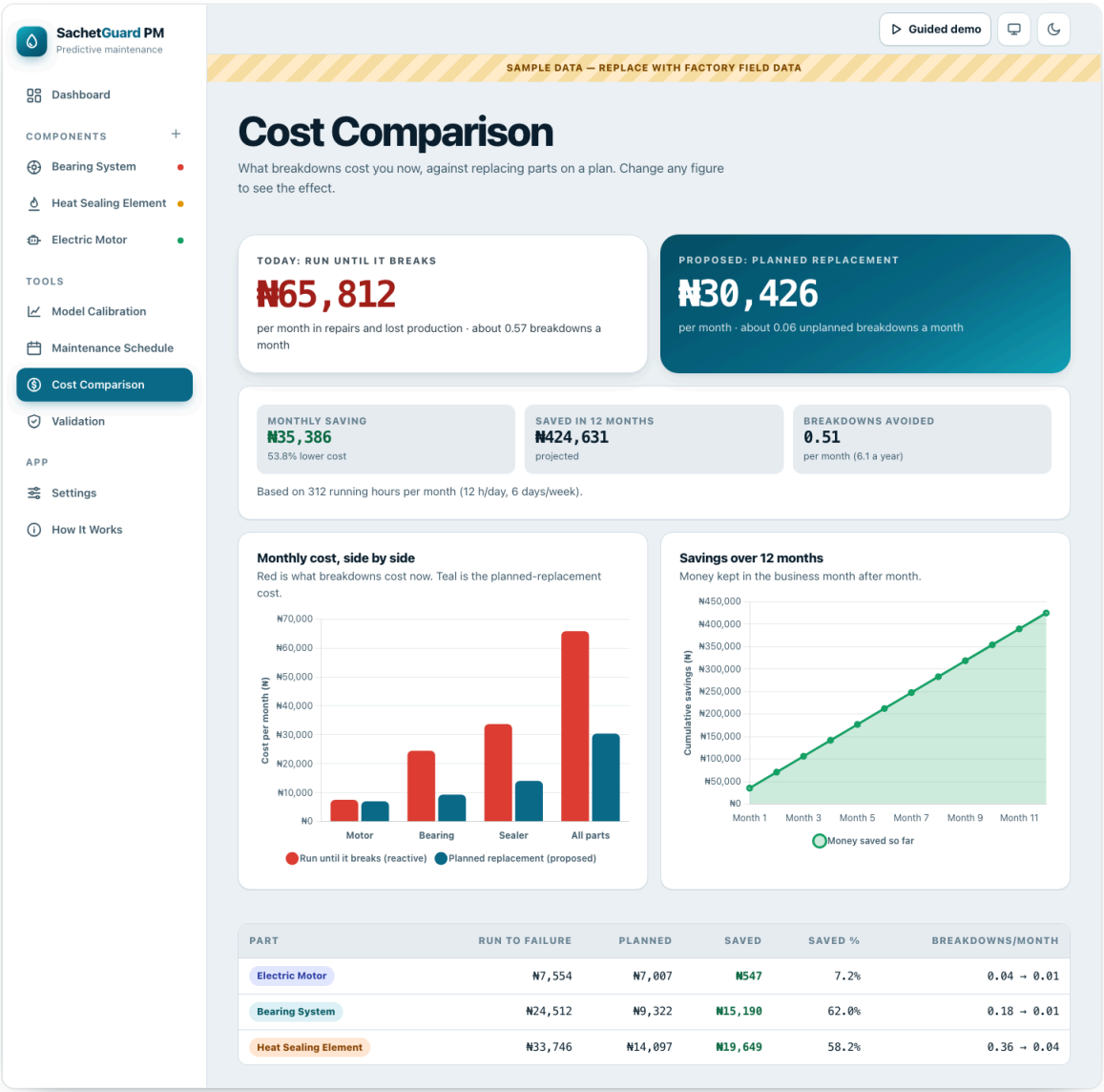
WHAT IT SHOWS

- A **month-by-month** summary of jobs for each part. "replace" marks a month with a part replacement.
- A **task list** with the date, the part, the job and the type of job.

WHAT YOU CAN DO

- Show all parts or one part at a time.
- Press **Print schedule** for a clean black-and-white A4 copy to give to technicians.

3.6 Cost Comparison



Cost Comparison. Today's cost against the proposed plan.

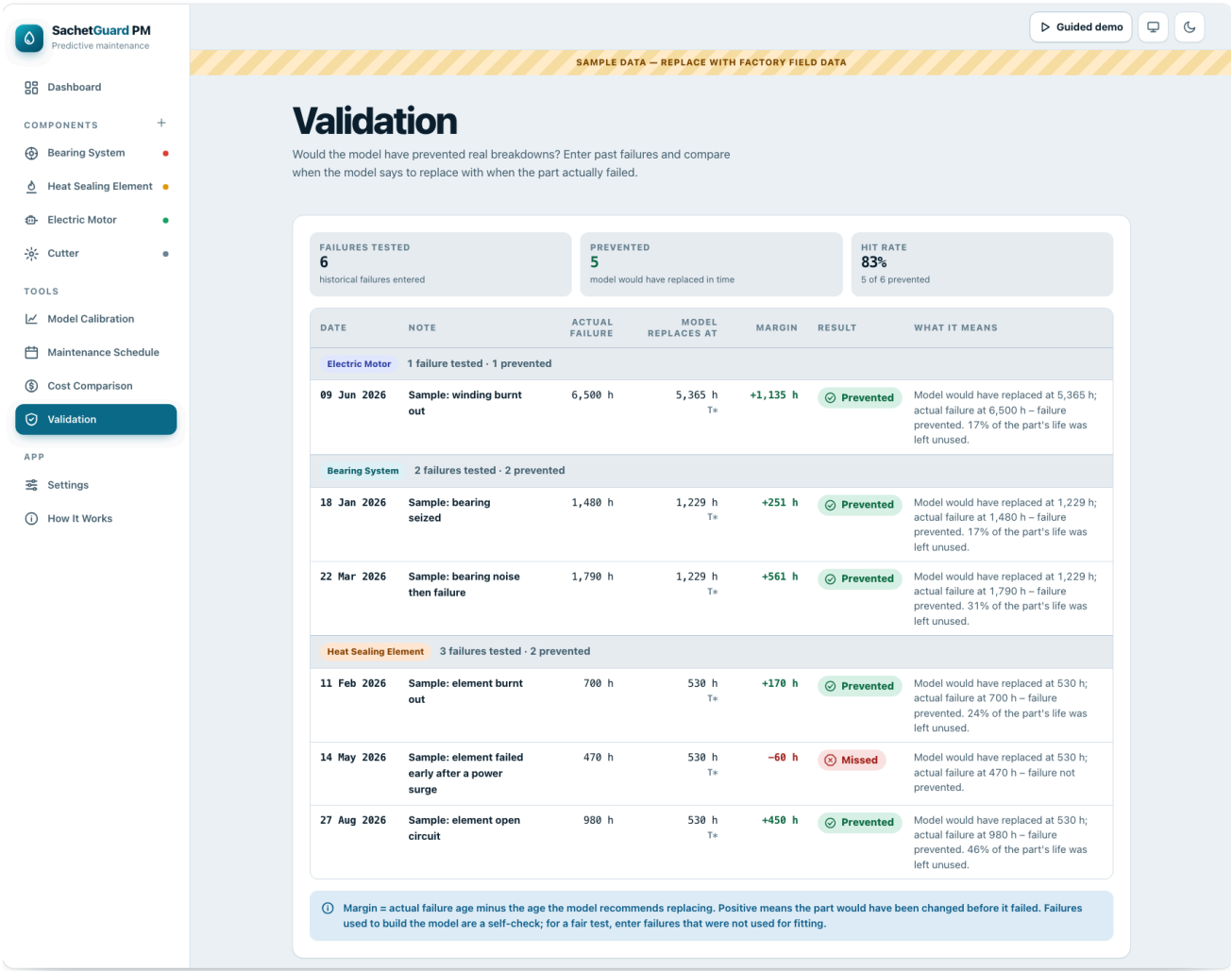
WHAT IT SHOWS

- The monthly cost of **running until breakdown** against the cost of **planned replacement**.
- Monthly saving, saving over 12 months, and breakdowns avoided.
- A bar chart per part and a line chart of savings building up over a year.

WHAT YOU CAN DO

- Change any cost: repair cost, downtime hours, lost production per hour, new part cost, labour and planned downtime.
- All results update straight away.

3.7 Validation

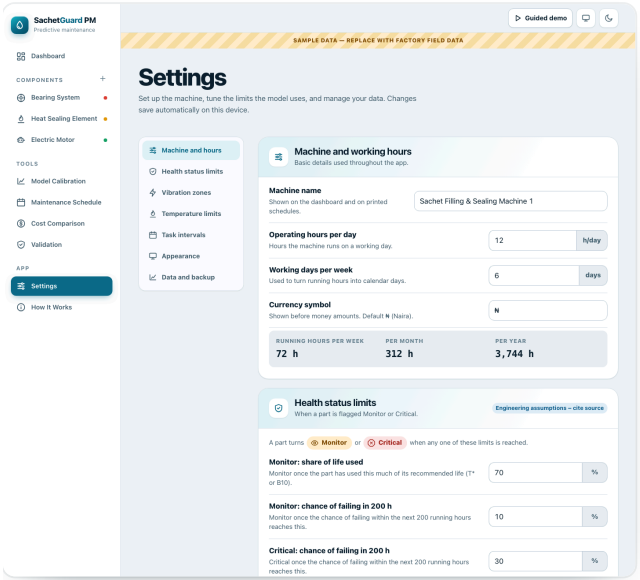


Validation. Checks the model against real past breakdowns.

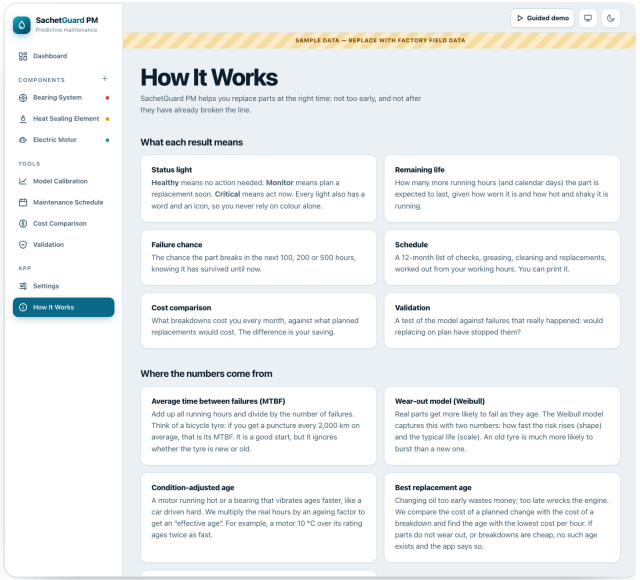
Here you enter past failures (the part, its age when it failed, and the date). For each one, the app compares **when the model would have replaced the part** with **when it really failed**.

- **Prevented** means the model would have replaced the part before it broke.
- **Missed** means the part broke before the model's replacement age.
- The **hit rate** is the share of failures that would have been prevented. With the sample data it is **5 of 6 (83%)**.

3.8 Settings and 3.9 How It Works



Settings.



How It Works.

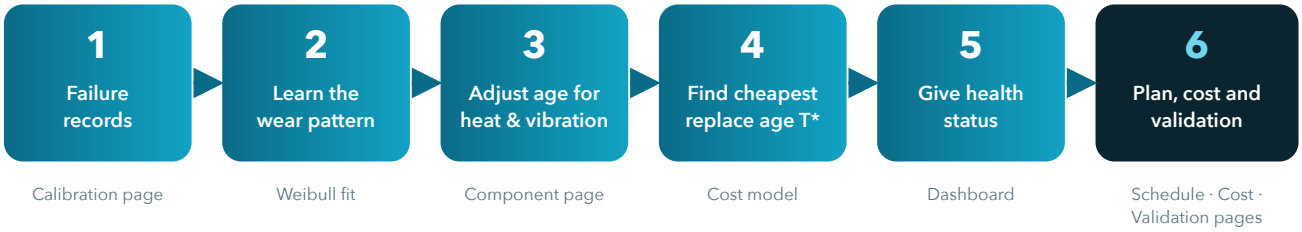
Settings holds everything that applies to the whole factory:

- Machine name, **operating hours per day** and **working days per week**. These turn running hours into calendar days.
- **Health status limits**: when a part turns Monitor or Critical.
- **Vibration zones** and **temperature limits** for each type of part.
- Routine task intervals, appearance, and **data and backup** (back up, restore, load sample data, reset all data).

How It Works explains every result and formula in plain words, and lists the limits of the model. It is a good page to show an examiner.

04 How the app thinks

Behind every screen, the app follows the same six steps. Each step uses a standard method from reliability engineering.



Step 1 – Collect failure records

For each part, we write down the running hours at which it failed. A part that was removed while still working is a **suspension**. It still tells us something: the part lasted at least that long.

Step 2 – Learn the wear pattern (Weibull model)

The app fits a **Weibull model** to the records. This model is described by two numbers:

- **η (eta), the scale**: the typical life. About 63% of parts have failed by this age.
- **β (beta), the shape**: how the part fails. β below 1 means early failures. β equal to 1 means random failures. **β above 1 means the part wears out**, so planned replacement makes sense.

The method used is **Median Rank Regression**: the records are put in order, each gets a "rank", and a straight line is drawn through them on a special graph. The slope of the line gives β. The R^2 value shows how well the line fits (1.0 is perfect).

Step 3 – Adjust the age for heat and vibration

A part that runs hot or shakes a lot wears out faster. So the app multiplies the real age by two factors to get the **adjusted (effective) age**:

- **Temperature factor.** For the motor, every 10 °C above the rated temperature doubles the wear (the "10-degree rule"). For the bearing, warning and alarm temperatures add extra wear. For the sealing element, every degree away from its set temperature adds a little wear.
- **Vibration factor.** The vibration reading is put into zone A, B, C or D (ISO 10816 style). Zone A adds nothing; zone D doubles the wear. Vibration is used for the motor and the bearing only.

Step 4 – Find the cheapest replacement age, T*

A planned replacement costs less than a breakdown. The app works out the **average cost per running hour** for every possible replacement age, and picks the age with the **lowest** cost. This is **T***. If there is no clear best age (for example when β is 1 or less), the app uses the **B10 life** instead: the age by which 1 in 10 parts fail.

Step 5 – Give each part a health status

STATUS	WHEN IT HAPPENS (DEFAULT SETTINGS)
Critical	Any one of: the adjusted age has reached T*; the chance of failing in the next 200 h is 30% or more; vibration is in zone D; or temperature is over the alarm limit.
Monitor	Not critical, but the part has used 70% or more of its replacement life, or the chance of failing in the next 200 h is 10% or more.
Healthy	None of the above.

All these limits can be changed in Settings.

Step 6 – Build the plan, the cost comparison and the validation

The replacement age and the routine task intervals are turned into **calendar dates** using the factory's working hours. The cost of the plan is compared with the cost of waiting for breakdowns. Finally, past failures are checked to see whether the plan would have prevented them.

The main formulas

QUANTITY	FORMULA	IN PLAIN WORDS
MTBF	total hours ÷ number of failures	Average hours between failures.
Reliability (Weibull)	$R(t) = \exp[-(t/\eta)^{\beta}]$	Chance the part is still working at age t.
Hazard rate	$h(t) = (\beta/\eta)(t/\eta)^{\beta-1}$	Risk of failing in the next hour, at age t.
Mean life (MTTF)	$\eta \cdot \Gamma(1 + 1/\beta)$	Average life from the model.
B10 life	$\eta \cdot (-\ln 0.90)^{1/\beta}$	Age by which 10% of parts fail.
Adjusted age	$t_{eff} = t \cdot k_{temp} \cdot k_{vib}$	Real age, sped up by heat and vibration.
Chance of failing soon	$P = 1 - R(t+\Delta t) \div R(t)$	Chance of failing in the next Δt hours, given it works now.
Remaining life	$\int_t^{\infty} R(x)\,dx \div R(t)$	Expected running hours left.
Cost per hour	$C(T) = [Cp \cdot R(T) + Cf \cdot (1-R(T))] \div \int_0^T R\,dt$	Average cost per running hour if replaced at age T. Its lowest point is T*.

Cp = cost of a planned replacement (part + labour + short stop). Cf = cost of a breakdown (repair + lost production during the stop).

05 Results with the sample data

These are the numbers the app produces from the sample failure records. They show the model working from start to finish. They must be replaced with real factory data before final conclusions are drawn.

PART	B (SHAPE)	H (SCALE)	R^2	T*	ADJUSTED AGE NOW	STATUS
Electric motor	5.71	7,129 h	0.969	5,365 h	2,200 h	Healthy
Bearing system	7.93	1,817 h	0.962	1,229 h	1,848 h	Critical
Heat sealing element	4.48	960 h	0.991	530 h	400 h	Monitor

What these numbers mean

- All three β values are well above 1. This means **all three parts wear out with age**, so planned replacement is the right strategy.
- All R^2 values are above 0.96. The Weibull model **fits the data very well**.
- The **bearing** is only 1,100 hours old, but it runs hot (72 °C) and vibrates in zone C. Its adjusted age is 1.68 times higher: 1,848 h. That is past its best replacement age, so the app says **"Replace the bearing now"**.
- The **sealing element** is at 400 h out of 530 h. The app says **plan to replace it within about 13 days**.
- The **motor** is healthy. Its next replacement is about **308 days** away.

Money and validation

₦65,812 / month
Cost today, waiting for breakdowns (about 0.57 breakdowns a month).

₦30,426 / month
Cost with planned replacement (about 0.06 breakdowns a month).

₦35,386 saved
Each month, a 53.8% saving. About ₦424,631 in 12 months.

In the validation check, the model would have **prevented 5 of 6** past failures (83%). The one it missed was a sealing element that failed early after a power surge. This kind of failure is random and cannot be predicted from age.

> **Important:** the costs are realistic estimates for a small Makurdi plant, and the failure hours are sample values. Say this clearly in the write-up, and replace them with the factory's own figures where possible.

06 How the code is put together

The app is built with only the three basic web languages. **HTML** gives the structure of the page, **CSS** gives the look, and **JavaScript** does the calculations and makes the page interactive. No extra framework is used. The one outside library is **Chart.js** (version 4.5.1), which draws the charts. A copy is kept inside the project, so no internet is needed.

The parts of the program

index.html – the page frame. Loads the styles (css/) and then the scripts below, in order.

WHAT YOU SEE

ui.js builds every page

charts.js draws the charts

THE THINKING

model.js status, plan, cost

math.js pure formulas

THE MEMORY

store.js saves and loads data

browser storage · backups

app.js – starts everything: loads saved data, draws the first page and listens for page changes.

tests/ – 69 checks of the maths

tools/bundle.js – makes the one-file version

Every file and its job

FILE	LINES	WHAT IT DOES
`index.html`	95	The frame of the app: the side menu, top bar, phone menu, footer and the empty area where pages appear. It loads all the other files.
`css/tokens.css`	105	The colour, font and spacing "tokens" for the light and dark themes. Change a colour here and it changes everywhere.
`css/base.css`	139	The general layout: side menu, top bar, page area, phone view and basic text styles.
`css/components.css`	588	The look of every building block: cards, buttons, input boxes, tables, status pills, gauges and charts.
`css/print.css`	38	A clean black-on-white layout used when the schedule is printed.
`js/math.js`	448	The reliability mathematics only: MTBF, exponential and Weibull formulas, the gamma function, B10 life, Weibull fitting, integration and the search for T*. Each function has a comment showing its equation.

`js/model.js`	687	The engineering model. It holds the part types and default settings, works out the temperature and vibration factors, analyses each part (status, remaining life, recommendation), builds the 12-month schedule, the cost comparison, the validation and the step-by-step working.
`js/store.js`	450	The data memory. It holds the sample data, saves everything in the browser, adds and deletes parts, records and tasks, imports and exports CSV files, and makes and restores backups.
`js/charts.js`	274	Draws all charts with Chart.js: reliability, hazard, cost curve, Weibull plot, failure chance and cost charts. Adds the "You are here" and "Replace here" markers.
`js/ui.js`	1,471	The user interface. It builds the HTML of every page, checks what the user types, reacts to clicks, runs the guided demo, presentation mode, theme switch and phone menu, and redraws results when data changes.
`js/app.js`	23	The starting point. It loads the stored data, prepares the screen and opens the dashboard.
`vendor/chart.umd.min.js`	–	Chart.js 4.5.1, the outside charting library (MIT licence). Not written for this project.
`tests/tests.html`, `tests.js`	65 + 406	69 unit tests. Each compares a result with a value worked out by hand or a known constant, and shows PASS or FAIL.
`tools/bundle.js`	36	A small Node.js script that joins all files into one: `dist/sachetguard-standalone.html`.

How the files work together

All JavaScript files share one object called `PM`. Each file adds its own part: `PM.math`, `PM.model`, `PM.store`, `PM.charts` and `PM.ui`. Because they are loaded in a fixed order, each file can use the ones loaded before it. This also lets the app run by simply double-clicking `index.html`, with no web server.

What happens when you type a new reading

1. You type, for example, a new temperature on a component page.
2. `ui.js` checks the number is sensible and passes it to `store.js`.
3. `store.js` saves it in the browser and tells the screen that data has changed.
4. `model.js` analyses the part again, using the formulas in `math.js`.
5. `ui.js` and `charts.js` redraw the status, numbers and charts. This all takes a fraction of a second.

How pages are shown

The app is a **single page**. The address ends with a short code such as `#/dashboard` or `#/component/bearing`. When it changes, `ui.js` reads it and builds the matching page in the main area. This makes the browser's Back button work as expected.

How the work was checked

Open `tests/tests.html` in a browser. It runs **69 tests** on the calculation engine. For example, it checks that MTBF = 1000 ÷ 4 = 250, that Γ(5) = 24, and that the Weibull fit gives back the right β and η from known data. All 69 tests pass. A screenshot of this page is good evidence for the report.

Accessibility and good practice

- Works with the keyboard alone, with a visible focus outline and a "Skip to content" link.
- Screen readers are told when results change.
- Respects the "reduce motion" setting of the computer.
- Every number typed is checked, and a clear message is shown if it is not valid.

07 Entering real factory data

Follow these steps to replace the sample data with the factory's own figures.

1. Open **Settings**. Enter the machine name, the operating hours per day and the working days per week.
2. Press **Reset all data** to start empty, or keep the sample and type over it.
3. Add any extra parts with **Add component**. Choose a name and a type (motor, bearing, heat element or other). The type sets the temperature rule and the routine tasks.
4. Open **Model Calibration**. Choose a part and add one record for each failure: the running hours of the part when it failed. Mark parts that did not fail as **suspensions**. You can also **Import CSV** with the columns `hours,type`.
5. Press **Fit Weibull**. At least 3 failures are needed. If you have fewer, type β and η by hand from a maker's data sheet or an earlier study.
6. Open **Cost Comparison** and enter the plant's real costs: repair cost, downtime hours, lost production per hour, new part cost and labour.
7. Open each part's page and enter today's readings: hours since replacement, temperature, vibration and last maintenance date.
8. Open **Validation** and enter past failures that were **not** used for fitting, for a fair test.
9. Press **Settings → Back up data** to save a copy of everything.

> **Check the limits.** The vibration and temperature limits in Settings are marked "engineering assumptions – cite source". Check them against the machine manuals, and cite the standard (for example ISO 10816 for vibration) in the report.

08 Tips for the defence

Short, simple answers to questions an examiner is likely to ask.

Why did you use the Weibull model?

It is the standard model for mechanical parts that wear out. Its shape value β tells us whether failures are early, random or caused by wear. The sample bearing data gives β ≈ 7.9, which is strong wear-out, so replacing at a fixed age is justified. The app also draws the simpler exponential (constant risk) model for comparison.

How did you find the best replacement age?

With the age-replacement cost model. For every possible replacement age, it works out the average cost per running hour, counting both planned replacements and breakdowns. The age with the lowest cost is T*. The app searches with a grid first, then refines with the golden-section method.

Why do you adjust the age for temperature and vibration?

Because harsh conditions make parts wear faster. A motor running 10 °C above its rated temperature ages about twice as fast (the 10-degree rule for insulation). Vibration zones follow ISO 10816-style limits.

How do you know the calculations are correct?

There are 69 unit tests that compare the program's results with hand calculations and known values. All pass. Also, every result in the app can be opened with "How was this calculated?", which shows the formula with the real numbers.

How do you know the model works in practice?

The Validation page compares the recommended replacement age with real past failures. With the sample data, 5 of 6 failures would have been prevented.

Why a web app and not Excel or MATLAB?

A browser is on every computer and phone. The app needs no licence, no installation and no internet, so a small factory can use it straight away. It also gives plain-language advice that technicians can act on.

How to show it live

Open the app, press **Presentation mode**, then **Guided demo**. On a component page, change the hours, temperature or vibration and show that the status, charts and advice change at once.

09 Limits of the work and glossary

Limits to state honestly

- The number of failure records is small. More records will give more reliable β and η values.
- The temperature and vibration factors and the status limits are engineering assumptions. They are reasonable, but they should be checked against the maker's data.
- The model predicts wear-out failures. It cannot predict random events such as power surges.
- Data is stored in one browser on one device. Backups must be made by hand.
- Readings are typed in by hand. A future version could read sensors directly.

Glossary

Adjusted (effective) age – real running hours multiplied by the heat and vibration factors.

B10 life – the age by which 10% of parts are expected to fail.

β (beta), shape – tells how a part fails. Above 1 means it wears out.

η (eta), scale – the typical life; 63% of parts have failed by this age.

Failure record – the running hours at which a part broke.

Hazard rate – the risk of failing in the next hour.

MTBF – mean time between failures: average hours between breakdowns.

MTTF – mean time to failure: the average life from the Weibull model.

Median Rank Regression – the method used to fit β and η by drawing a best straight line through ranked failure data.

Predictive maintenance – deciding when to service a part from its condition and failure history.

Preventive maintenance – replacing or servicing a part on a plan, before it fails.

R^2 (fit quality) – how close the data is to the straight line; 1.0 is perfect.

Reactive maintenance – waiting for a part to break, then fixing it.

Reliability R(t) – the chance a part is still working at age t.

Remaining useful life – the expected running hours left before failure.

Suspension – a part that was removed, or is still running, without failing.

T* – the best (cheapest) replacement age.

Weibull model – the standard formula used to describe how long parts last.